

SEGUNDO TRABAJO PRÁCTICO EVALUATIVO

producción Web | Escuela Da Vinci

Prof. Carlos Ferrer | Puntuación: 1 a 10 | Individual o en grupo

1. Consigna general

Continuar y extender el proyecto del primer parcial (*mismo tema, mismo repositorio de base*). El sitio evoluciona incorporando un **panel de administración protegido, autenticación y roles de usuario, ABM del blog con imágenes y validación de formularios**.

El carrito de compras no se implementa en este parcial, pero el sitio debe seguir estructurado pensando en su incorporación futura: el producto/servicio debe estar presente y bien definido, ya que será la base sobre la que se trabajará en la próxima entrega.

2. Arquitectura del sitio

El sitio se divide en dos partes claramente separadas:

2.1 Sitio público (frontend)

- **Home** — Presentación del producto o servicio.
- **Blog / Novedades / Noticias** — Entradas cargadas dinámicamente desde la BD.
- **Registro y login de usuarios** — Creación de cuenta, inicio y cierre de sesión.

2.2 Panel de administración (backend)

- **Acceso restringido** — Solo usuarios con rol administrador pueden ingresar. Verificado mediante un Middleware personalizado.
- **ABM del blog** — Alta, baja y modificación de entradas, incluyendo upload y cambio de imagen por entrada.
- **Listado de usuarios** — Vista con todos los usuarios registrados y detalle individual que muestre el servicio contratado o las compras realizadas.

3. Base de datos

- **Mínimo 3 tablas:** users + tabla de blog/noticias + tabla de servicios/compras (u otra que modele la relación usuario ↔ producto).
- **La tabla de blog/noticias** debe tener mínimo 5 campos propios (sin contar id ni timestamps).
- **Debe haber relaciones entre tablas** modeladas con Eloquent (hasMany, belongsTo, belongsToMany, etc.).
- **Toda la estructura** se crea con Migrations.

- **La carga inicial de datos** se realiza con Seeders, incluyendo al menos un usuario con servicio contratado.

4. Stack tecnológico requerido

4.1 PHP / Laravel

- **Laravel (versión vista en clase)** con arquitectura MVC estricta.
- **Middleware personalizado** para verificar el rol administrador en las rutas del panel admin (no alcanza con el middleware auth de Laravel).
- **Validación de todos los formularios** con Form Requests o validate() en el controller, con retroalimentación de errores en la vista.
- **Modelos relacionados** que reflejen las relaciones de la BD mediante métodos Eloquent.
- **Upload de imágenes** para las entradas del blog (guardar en storage o public/uploads).
- **Dos roles de usuario:** usuario común y administrador.

4.2 Vistas

- **Motor Blade obligatorio** en todas las vistas.
- **HTML semántico correcto** en el sitio público y en el panel admin.
- **CSS personalizado o framework** (Bootstrap, Tailwind, Bulma). El admin puede tener un estilo diferente al sitio público.

5. Criterios de puntuación

La nota se compone de **5 puntos base (condiciones mínimas)** + hasta **5 puntos adicionales** por calidad de implementación. Mismo criterio que el primer parcial.

Criterio	Puntos	Notas para el corrector
CONDICIONES MÍNIMAS (nota 4)		
Home con presentación real del producto/servicio (no solo bienvenida vacía)	0,5 pt	Verificar que tenga contenido descriptivo real — igual que en el 1er parcial
Sección de blog/novedades/noticias con entradas cargadas dinámicamente desde la BD	0,5 pt	¿Las entradas vienen de la BD? ¿No son hardcodeadas en el Blade?
Registro y autenticación de usuarios (sign up + login + logout funcionales)	0,5 pt	Probar crear cuenta, iniciar sesión y cerrar sesión
Panel de administración protegido: requiere login de usuario con rol administrador	0,5 pt	Intentar acceder a /admin (o ruta equivalente) sin login — debe redirigir

ABM completo del blog/novedades desde el admin (alta, baja, modificación)	0,5 pt	Probar crear, editar y eliminar una entrada desde el panel
ABM de imágenes en el blog (upload y modificación de imagen por entrada)	0,5 pt	Probar subir y cambiar una imagen — verificar que se guarda en storage o public
Listado de usuarios en el admin con detalle individual (servicio contratado o compras)	0,5 pt	Verificar que el detalle muestre la relación usuario ↔ servicio/compra
Dos roles implementados (usuario común y administrador) con Middleware personalizado para verificar el rol admin	0,5 pt	Buscar en /app/Http/Middleware: ¿existe middleware propio (no solo auth de Laravel)?
Validación de todos los formularios con retroalimentación de errores al usuario	0,5 pt	Probar enviar formularios vacíos y con datos inválidos — ¿aparecen mensajes de error?
Mínimo 3 tablas con Migrations, Seeders y al menos 1 usuario con servicio contratado cargado por Seeder	0,5 pt	Verificar /database/migrations y /database/seeders. Correr php artisan migrate:fresh --seed y confirmar datos
SUBTOTAL MÍNIMOS	5 pts	Base para aprobar (nota 4)
ADICIONALES (hasta nota 10)		
Complejidad destacable: funcionalidad que va más allá del enunciado mínimo	1 pt	¿Tiene lógica de negocio propia? ¿Algo no trivial que el alumno decidió agregar?
Accesibilidad, navegabilidad y usabilidad del sitio (roles bien diferenciados, UX clara)	0,5 pt	¿El flujo usuario/admin es intuitivo? ¿Hay feedback visual para acciones?
Tablas de relación adicionales en la BD (más allá de las 3 mínimas, con relaciones Eloquent correctas)	0,5 pt	Revisar modelos: ¿definen hasMany, belongsTo, belongsToMany, etc.?
Buenas prácticas OOP: SRP, controllers delgados, lógica en Models o Services, nomenclatura coherente	0,5 pt	¿Los controllers tienen métodos largos? ¿Hay lógica en web.php? ¿Los nombres son consistentes?
Documentación con PHPDoc en clases y métodos (Models, Controllers, Middleware)	0,5 pt	Buscar bloques /** ... */ con @param, @return, @throws donde corresponda
Estilización y HTML semántico de calidad en ambas partes (sitio público y admin)	0,5 pt	¿El CSS es propio o de framework? ¿El HTML usa etiquetas semánticas correctas?
Prolijidad del código y organización del proyecto (indentación, sin código muerto, assets ordenados)	0,5 pt	¿Hay variables sin usar, bloques comentados, archivos sueltos fuera de su carpeta?
SUBTOTAL ADICIONALES	5 pts	Suma a los 5 mínimos → máximo 10 pts

6. Descuentos y penalizaciones

Los descuentos son acumulativos. El corrector debe ejecutar el proyecto antes de evaluar el código.

Error / Infracción	Descuento	Criterio de evaluación
DESAPROBACIÓN AUTOMÁTICA		
El proyecto no corre (dependencias rotas, migraciones con error, .env ausente)	Nota 1	Ejecutar: composer install → php artisan migrate --seed → php artisan serve
No usa Laravel o usa una versión distinta a la vista en clase	Nota 1	Verificar composer.json: laravel/framework en require
El panel admin no tiene protección real (cualquier usuario puede acceder sin autenticación)	Nota 1	Acceder a /admin sin sesión activa — debe redirigir al login
Trabajo copiado (total o parcialmente) — tanto copiador como copiado	Nota 1	A discreción del docente: puede incluir preguntas orales para verificar autoría
Tema sobre política o religión	Nota 1	
DESCUENTOS POR INCUMPLIMIENTOS TÉCNICOS		
Tablas creadas sin Migrations	-1 pt	Verificar /database/migrations: ¿existen archivos para cada tabla?
Datos iniciales cargados sin Seeders	-1 pt	Verificar /database/seeders y que DatabaseSeeder los invoque
Middleware de verificación de rol admin ausente o usa solo el middleware auth de Laravel sin uno propio	-1 pt	Buscar en /app/Http/Middleware: debe existir un middleware personalizado registrado en el kernel o en bootstrap/app.php
Formularios sin validación o sin retroalimentación de errores	-1 pt	Probar enviar formularios vacíos — ¿aparecen mensajes de error en la vista?
Upload de imágenes ausente o no funcional en el ABM del blog	-0,5 pt	Probar subir una imagen desde el panel admin — verificar que se guarda y se muestra
Modelos sin relaciones Eloquent (sin hasMany, belongsTo, etc.) pese a existir relaciones en la BD	-0,5 pt	Revisar /app/Models: ¿los modelos definen sus relaciones?
Vistas sin Blade (PHP mezclado con HTML crudo, sin directivas @)	-1 pt	Buscar archivos .blade.php y presencia de {{ }}, @foreach, @if, @extends, etc.
Menos de 3 tablas en la BD o tabla de blog con menos de 5 campos propios	-0,5 pt	Contar tablas en /database/migrations y columnas en la migración del blog

DESCUENTOS POR HTML SEMÁNTICO EN VISTAS		
1 a 3 errores (abuso de <div>, como separador, jerarquía de headings incorrecta, /<i> en lugar de /)	-0,5 pt	Revisar las vistas Blade principales del sitio público y del admin
4 a 7 errores semánticos en vistas	-1 pt	
8 o más errores semánticos en vistas	-2 pts	Implica pérdida del punto de semántica HTML
CONDICIONES DE ENTREGA		
Incumplimiento de formato de nombre del ZIP o datos.txt incompleto	-1 pt	Verificar nombre del archivo y todos los campos de datos.txt

7. Checklist de verificación rápida para el corrector

Secuencia recomendada antes de revisar el código:

- 1. `composer install` — ¿instala sin errores?
- 2. Copiar `.env.example` → `.env`, configurar BD y ejecutar `php artisan key:generate`
- 3. `php artisan migrate:fresh --seed` — ¿migra y carga datos sin errores?
- 4. `php artisan serve` — ¿levanta el servidor?
- 5. Registrar un usuario común → verificar que no puede acceder al panel admin
- 6. Iniciar sesión como administrador → verificar acceso al panel y que el ABM funcione
- 7. Probar upload de imagen en el blog → verificar que se guarda y se muestra
- 8. Enviar un formulario vacío → verificar mensajes de error en la vista

8. Modalidad y condiciones de entrega

8.1 Nombre del archivo

Individual:

`apellido-nombre.[zip|rar]`

En pareja:

`apellido-nombre_apellido2-nombre2.[zip|rar]`

Ejemplos: `ferrer-carlos.zip` / `ferrer-carlos_garcia-ana.zip`

8.2 Contenido del ZIP

- Proyecto Laravel completo
- `.env.example` configurado con los datos de conexión

- `datos.txt` con los datos completos del o los estudiantes

8.3 Archivo `datos.txt`

- Escuela de Arte Multimedial Da Vinci
- CARRERA: [carrera]
- MATERIA: Portales y Comercio Electrónico
- TRABAJO PRÁCTICO: Segundo Parcial
- PROFESOR: Carlos Ferrer
- ALUMNO/S: [apellido, nombre — uno por línea si son dos]
- CUATRIMESTRE: [cuatrimestre]
- AÑO: [año]
- TURNO: [turno]
- COMISIÓN: [comisión]

9. Consideraciones finales

- La entrega se realiza a través de Campus en la fecha publicada. No se aceptan entregas fuera de término ni por email.
- El incumplimiento de las condiciones de entrega descuenta al menos 1 punto.
- El docente puede realizar preguntas orales sobre el código entregado para verificar la autoría del trabajo.
- **Trabajo copiado:** tanto copiador como copiado reciben nota 1 automáticamente.